LeccionSVM

November 4, 2026

[1]:
```
import pandas as pd
import pylab as pl
import numpy as np
import scipy.optimize as opt
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
import os
cell_df = pd.read_csv("cell_samples.csv")
cell_df
```

[2]:

	ID	Clump	UnifSize	UnifShape	MargAdh	SingEpiSize	BareNuc	BlandChrom	NormNucl	Mit	Class
0	1000025	5	1	1	1	2	1	3	1	1	2
1	1002945	5	4	4	5	7	10	3	2	1	2
2	1015425	3	1	1	1	2	2	3	1	1	2
3	1016277	6	8	8	1	3	4	3	7	1	2
4	1017023	4	1	1	3	2	1	3	1	1	2
..	…	…	…	…	…	…	…	…	…	…	…
694	776715	3	1	1	1	3	2	1	1	1	2
695	841769	2	1	1	1	2	1	1	1	1	2
696	888820	5	10	10	3	7	3	8	10	2	4
697	897471	4	8	6	4	3	4	10	6	1	4
698	897471	4	8	8	5	4	5	10	4	1	4

```
697          10         6    1      4
698          10         4    1      4

[699 rows x 11 columns]
```

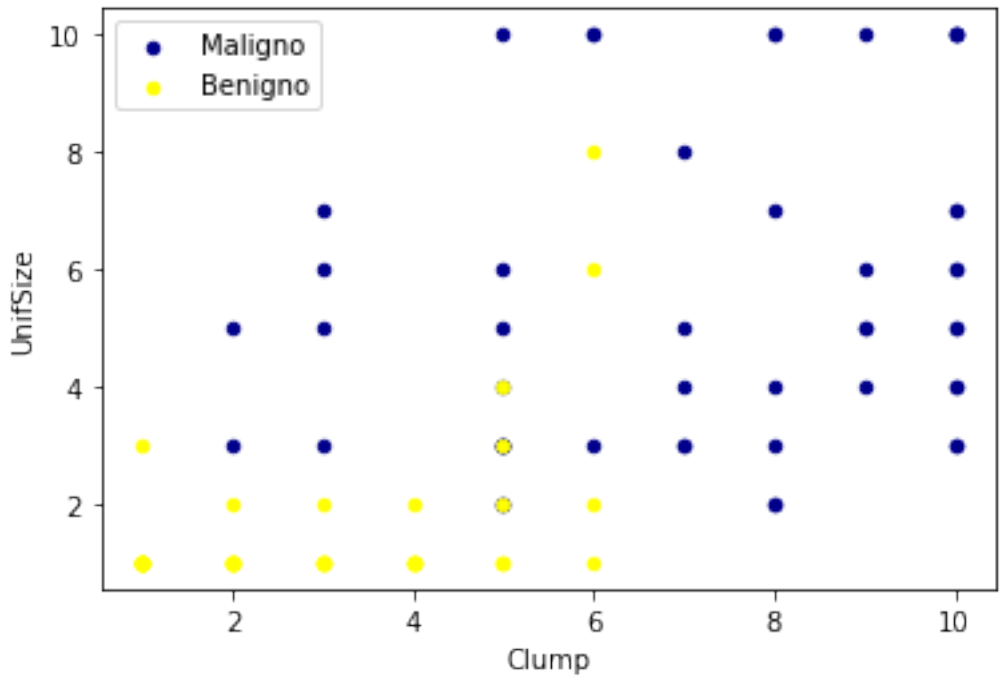
[3]:
```
cell_df["Class"] = cell_df["Class"].replace({2:0})  # Tumor Benigno
cell_df["Class"] = cell_df["Class"].replace({4:1})  # Tumor Maligno
cell_df
```

[3]:
```
           ID  Clump  UnifSize  UnifShape  MargAdh  SingEpiSize BareNuc  \
0     1000025      5         1          1        1            2       1
1     1002945      5         4          4        5            7      10
2     1015425      3         1          1        1            2       2
3     1016277      6         8          8        1            3       4
4     1017023      4         1          1        3            2       1
..        …    …       …          …       …            …     …
694    776715      3         1          1        1            3       2
695    841769      2         1          1        1            2       1
696    888820      5        10         10        3            7       3
697    897471      4         8          6        4            3       4
698    897471      4         8          8        5            4       5

     BlandChrom  NormNucl  Mit  Class
0             3         1    1      0
1             3         2    1      0
2             3         1    1      0
3             3         7    1      0
4             3         1    1      0
..          …       …  …    …
694           1         1    1      0
695           1         1    1      0
696           8        10    2      1
697          10         6    1      1
698          10         4    1      1

[699 rows x 11 columns]
```

[5]:
```
ax = cell_df[cell_df["Class"] == 1][0:50].plot(kind="scatter", x="Clump",␣
 ↪y="UnifSize", color="DarkBlue", label="Maligno")
ax = cell_df[cell_df["Class"] == 0][0:50].plot(kind="scatter", x="Clump",␣
 ↪y="UnifSize", color="Yellow", label="Benigno", ax=ax)
plt.show()
```



[6]:
```
# Pre-Procesamiento de datos y selección
cell_df.dtypes
```

[6]:
```
ID              int64
Clump           int64
UnifSize        int64
UnifShape       int64
MargAdh         int64
SingEpiSize     int64
BareNuc        object
BlandChrom      int64
NormNucl        int64
Mit             int64
Class           int64
dtype: object
```

[7]:
```
# Eliminación de renglones no numéricos en BareNuc
cell_df = cell_df[pd.to_numeric(cell_df['BareNuc'], errors='coerce').notnull()]
cell_df['BareNuc'] = cell_df['BareNuc'].astype('int')
cell_df.dtypes
```

[7]:
```
ID              int64
Clump           int64
UnifSize        int64
UnifShape       int64
```

```
MargAdh         int64
SingEpiSize     int64
BareNuc         int64
BlandChrom      int64
NormNucl        int64
Mit             int64
Class           int64
dtype: object
```

[14]:
```
feature_df = cell_df[["Clump", "UnifSize", "UnifShape", "MargAdh",␣
 ↪"SingEpiSize", "BareNuc", "BlandChrom", "NormNucl", "Mit"]]
X = np.asarray(feature_df)
X[0:5]
```

[14]:
```
array([[ 5,  1,  1,  1,  2,  1,  3,  1,  1],
       [ 5,  4,  4,  5,  7, 10,  3,  2,  1],
       [ 3,  1,  1,  1,  2,  2,  3,  1,  1],
       [ 6,  8,  8,  1,  3,  4,  3,  7,  1],
       [ 4,  1,  1,  3,  2,  1,  3,  1,  1]])
```

[15]:
```
cell_df["Class"] = cell_df["Class"].astype("int")
y = np.asarray(cell_df["Class"])
y[0:5]
```

[15]:
```
array([0, 0, 0, 0, 0])
```

[16]:
```
# Creación de grupos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=4)
print("Grupo de entrenamiento", X_train.shape, y_train.shape)
print("Grupo de prueba", X_test.shape, y_test.shape)
```

```
Grupo de entrenamiento (546, 9) (546,)
Grupo de prueba (137, 9) (137,)
```

#Modelación con SVM

Opciones de Kernel (Transformaciones):

1. Linear
2. Polynomial
3. Radial Basis function (RBF)
4. Sigmoid

[17]:
```
from sklearn import svm
clf = svm.SVC(kernel="rbf")
clf.fit(X_train, y_train)
```

[17]:
```
SVC()
```

[19]:
```
# Predicción para la base de prueba
yhat = clf.predict(X_test)
yhat[0:5]
```

[19]:
```
array([0, 1, 0, 1, 0])
```

[20]:
```
# Creación de matriz de confusión
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(X)
cm = confusion_matrix(y_test, yhat)
cm
```

[20]:
```
array([[85,  5],
       [ 0, 47]])
```

[22]:
```
y_test
```

[22]:
```
array([0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0,
       1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 1])
```

[23]:
```
yhat
```

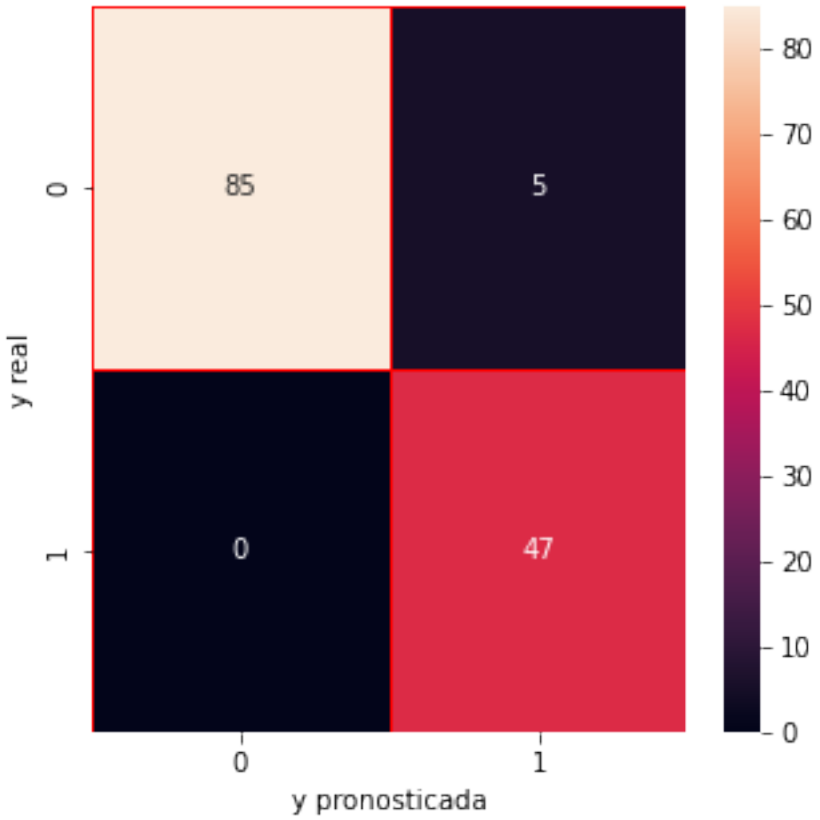
[23]:
```
array([0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0,
       1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 1,
       1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1,
       0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0,
       0, 0, 0, 0, 1])
```

[26]:
```
z = y_test - yhat
z
```

[26]:
```
array([ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0, -1, -1,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, -1,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0, -1,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0, -1,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0])
```

[28]:
```
# Visualización de la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize=(5,5))

sns.heatmap(cm, annot=True, linewidths=0.5, linecolor="red", fmt=".0f", ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y real")
plt.show()
```



1 Estadísticas de desempeño

[30]:
```
from sklearn.metrics import classification_report
cnf_matrix = confusion_matrix(y_test, yhat, labels=[0,1])
print(classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       1.00      0.94      0.97        90
           1       0.90      1.00      0.95        47
```

```
    accuracy                           0.96       137
   macro avg       0.95      0.97      0.96       137
weighted avg       0.97      0.96      0.96       137
```

2 Determinación de niveles de Precisión (Precision)

La Precisión ("Precision") = Porcentaje de predicciones correctas relativas al total de predicciones:

- El total se calcula por columna
- Se contesta a la pregunta: ¿Qué porcentaje de las veces que hacemos un pronóstico de cierto tipo éste es correcto?

[31]:
```
PrecisionBenigno = cm[0,0] / (cm[0,0] + cm[1,0])
PrecisionBenigno
```

[31]: 1.0

[32]:
```
PrecisionMaligno = cm[1,1] / (cm[1,1] + cm[0,1])
PrecisionMaligno
```

[32]: 0.9038461538461539

3 Determinación de niveles de Recuperación (Recall)

La Recuperación (Recall) = Porcentaje de predicciones correctas relativas al total de valores reales

- El total se calcular por renglón
- Se contesta a la pregunta: ¿Qué porcentaje de las veces que se tiene un valor es de identificarlo?

[33]:
```
RecallBenigno = cm[0,0] / (cm[0,0] + cm[0,1])
RecallBenigno
```

[33]: 0.9444444444444444

[34]:
```
RecallMaligno = cm[1,1] / (cm[1,0] + cm[1,1])
RecallMaligno
```

[34]: 1.0

4 Determinación del Score F1

F1 Score = Media armónica ponderada de la precisión y la recuperación. Cuanto más se acerque a 1, mejor será el modelo

- F1 Score = 2 * (Precision * Recall) / (Precision + Recall)

[35]:
```
F1Benigno = 2 * (PrecisionBenigno * RecallBenigno) / (PrecisionBenigno +␣
↪RecallBenigno)
F1Benigno
```

[35]: 0.9714285714285714

[36]:
```
F1Maligno = 2 * (PrecisionMaligno * RecallMaligno) / (PrecisionMaligno +␣
↪RecallMaligno)
F1Maligno
```

[36]: 0.9494949494949495

5 Determinación de soportes

Soporte = Número de observaciones que pertenecen en forma real a cada clase posible (Total por renglón)

[37]:
```
SoporteBenigno = cm[0,0] + cm[0,1]
SoporteBenigno
```

[37]: 90

[38]:
```
SoporteMaligno = cm[1,0] + cm[1,1]
SoporteMaligno
```

[38]: 47

6 Determinación de la Precisión Global (Accuracy)

Accuracy = Porcentaje de predicciones correctas

[39]:
```
Correctos = cm[0,0] + cm[1,1]
Incorrectos = cm[0,1] + cm[1,0]
PrecisionGlobal = Correctos / (Correctos + Incorrectos)
PrecisionGlobal
```

[39]: 0.9635036496350365

7 Promedios simples por indicador (Macro Average)

Promedio simples por Precisión, Recuperación y F1 Score

[40]:
```
MacroAvgPrecision = (PrecisionBenigno + PrecisionMaligno) / 2
MacroAvgPrecision
```

[40]: 0.9519230769230769

[41]:
```
MacroAvgRecall = (RecallBenigno + RecallMaligno) / 2
MacroAvgRecall
```

[41]: 0.9722222222222222

[42]:
```
MacroF1Score = (F1Benigno + F1Maligno) / 2
MacroF1Score
```

[42]: 0.9604617604617605

8 Promedios ponderados por indicador (Weighted Average)

Promedios ponderados de acuerdo a soportes por indicador

[43]:
```
PesoBenigno = SoporteBenigno / (SoporteBenigno + SoporteMaligno)
PesoMaligno = SoporteMaligno / (SoporteBenigno + SoporteMaligno)

WAvgPrecision = PrecisionBenigno * PesoBenigno + PrecisionMaligno * PesoMaligno
WAvgPrecision
```

[43]: 0.967012914093206

[44]:
```
WAvgRecall = RecallBenigno * PesoBenigno + RecallMaligno * PesoMaligno
WAvgRecall
```

[44]: 0.9635036496350364

[45]:
```
WAvgF1Score = F1Benigno * PesoBenigno + F1Maligno * PesoMaligno
WAvgF1Score
```

[45]: 0.9639038982104675

9 Prueba de SVM con Kernel Lineal

[46]:
```
clf = svm.SVC(kernel="linear")
clf.fit(X_train, y_train)

yhat = clf.predict(X_test)

# confusion matrix
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(X)
cm = confusion_matrix(y_test, yhat)
cm
```

[46]:
```
array([[85,  5],
       [ 0, 47]])
```

[47]:
```
from sklearn.metrics import classification_report
# import itertools

cnf_matrix = confusion_matrix(y_test, yhat, labels=[0,1])
# np.set_printoptions(precision=2)

print (classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       1.00      0.94      0.97        90
           1       0.90      1.00      0.95        47

    accuracy                           0.96       137
   macro avg       0.95      0.97      0.96       137
weighted avg       0.97      0.96      0.96       137
```

10 Prueba de SVM con Kernel Polinomial

[54]:
```
clf = svm.SVC(kernel="poly")
clf.fit(X_train, y_train)

yhat = clf.predict(X_test)

# confusion matrix
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(X)
cm = confusion_matrix(y_test, yhat)
cm
```

[54]:
```
array([[86,  4],
       [ 0, 47]])
```

[55]:
```
from sklearn.metrics import classification_report
# import itertools

cnf_matrix = confusion_matrix(y_test, yhat, labels=[0,1])
# np.set_printoptions(precision=2)

print (classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support
```

```
              0       1.00      0.96      0.98        90
              1       0.92      1.00      0.96        47

    accuracy                           0.97       137
   macro avg       0.96      0.98      0.97       137
weighted avg       0.97      0.97      0.97       137
```

11 Prueba de SVM con Kernel Sigmoide

[52]:
```
clf = svm.SVC(kernel="sigmoid")
clf.fit(X_train, y_train)

yhat = clf.predict(X_test)

# confusion matrix
from sklearn.metrics import confusion_matrix
y_pred = clf.predict(X)
cm = confusion_matrix(y_test, yhat)
cm
```

[52]:
```
array([[54, 36],
       [47,  0]])
```

[53]:
```
from sklearn.metrics import classification_report
# import itertools

cnf_matrix = confusion_matrix(y_test, yhat, labels=[0,1])
# np.set_printoptions(precision=2)

print (classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.53      0.60      0.57        90
           1       0.00      0.00      0.00        47

    accuracy                           0.39       137
   macro avg       0.27      0.30      0.28       137
weighted avg       0.35      0.39      0.37       137
```